Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 192 (2021) 3099–3108

Procedia Computer Science

www.elsevier.com/locate/procedia

25th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems

Eye Movement-Based Methods for Human-System Interaction. A Comparison of Different Approaches

Katarzyna Harezlak[a,*], Arkadiusz Duliban[a], Pawel Kasprowski[a]

[a]*Department of Applied Informatics, Silesian Univeristy of Technology, Akademicka 16, Gliwice, 44-100, Poland*

Abstract

The paper focuses on utilizing eye movements for controlling computer programs. Four different approaches were considered, including gazing at an element, blinking, and two solutions for eye gestures. The methods were implemented in a simple photo viewer. However, the developed functionalities can be utilized in other applications (switching e-book pages or on-screen keyboard elements). Various sizes for gazed components were verified, as well as the time required for performing particular actions, the number of unwanted element choices, and needed gesture repetitions. The methods were also assessed by the participants of the experiments. Based on the obtained results, it may be reasoned that components of size 200px are convenient for the application control. The users' opinions and the experiments' outcomes revealed that the gaze-based method and gestures based on joining points are satisfactory solutions.

© 2021 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of KES International.

Keywords: Human system interaction; eye movement; blinking, eye gestures, methods comparison;

1. Introduction

The sight has long been considered the most important of all human senses being a source of information for the brain. Based on eye movement, the analysis regarding people behavior, skills, intentions, or areas that attract their attention can be conducted [1, 2, 3, 4]. These possibilities result from the development of *eye-tracking*, a technique that allows recording the eye movement signal and calculating the users' *gaze point*. This signal is then subjected to further analysis and interpretation, according to the chosen purpose. Depending on the approach used, different information can be extracted from the same signal.

Eye-tracking has attracted attention for over 100 years. The solutions and experiences developed during this time were used when eye-tracking entered a new phase of development – interactive applications using the user's gaze. Until recently, the cost of purchasing a single eye tracker has been associated with a considerable expense [5], but

* Corresponding author. Tel.: +48-32-237-13-39
E-mail address: katarzyna.harezlak@polsl.pl

1877-0509 © 2021 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of KES International.
10.1016/j.procs.2021.09.082

many competitive, low-cost solutions have appeared lately. The presence of alternatives to professional solutions resulted in the spread of this technology type [6].

Among computer programs utilizing eye trackers, two types of applications can be distinguished. These that only examine eye movements received from the device are called *passive applications*; a representative example of which may be a program that analyzes and displays the reading pattern for a given text. The second group of applications is called *active applications*, and unlike passive ones, they allow a user to interact with the program and control it. These applications can still work with other input devices, such as the keyboard, mouse, and console [7]. In such a case, eye control will only be an additional means of interaction, facilitating certain operations. However, it can also be used to eliminate the need for other input devices. Such applications are controlled through eye gestures [8] and are equivalent to popular programs: word processors or calculators using on-screen keyboards. However, slightly more specialized software that allows scrolling web pages [9] or switching e-book pages can also be mentioned here.

2. The State of the Art

Computer system control is very often based on the analysis of fixations [5, 7, 10] described by (x, y) coordinates of the user's gaze as well as their start time and duration. The aforementioned creates many different possibilities for the implementation of the user-computer interaction.

2.1. Gaze-based Control

One of the proposed eye movement-based approaches for controlling the application is a method of selecting an object by a gaze [10, 11]. This approach is based on natural human behavior directing eyes to objects of interest. Actions such as clicking the mouse are performed by the user's eyes, which focus on the appropriate interface component [10, 12]. The use of gaze, instead of the traditional usage of the mouse or touchpad, can sometimes provide better results in terms of the initial speed [7, 12]. Focusing on a given point on the screen leads to the element choice, triggering a click action. When selecting an element, its dimensions play an important role. Research has shown that larger buttons require less time to be selected [10]; causing, however, their accidental selections [13].

A slightly different approach assumes that the user's interaction with the computer system through eyes should be as close as possible to traditional communication. There are solutions proposed simulating click events resembling the mouse work [12]. A more prolonged gaze on a given point on the screen saves its coordinates and marks the place where users want to perform the particular action. Then, a context menu is displayed, from which users – also with their eyes – choose the operation, and the application issues the appropriate "click" [5]. In this way, an emulated mouse often allows programs to perform most of the actions available using standard control methods. The main difficulty in such an approach is the necessity to assess whether the user actually wanted to initiate the action or his intentions were inadvertently understood by the application [13].

2.2. Interaction by Blinking Eyes

One of the alternatives to the previously described object selection is controlling the application by blinking or closing a single eye. This type of movement is mostly involuntary and serves mainly to lubricate the eye. However, it can also be performed in a controlled, and intentional manner [8, 10]. In one of the solutions proposed so far, eye blinking was used to confirm the willingness of the gaze-made choice [14]. The use of eye blink was also applied for constructing a more advanced solution in the form of a virtual on-screen keyboard that can be operated by closing the eyelids [15]. The designed keyboard allows not only to enter subsequent symbols in various types of programs but loading and navigating web pages as well.

Another solution allows using eye blink to initiate an action usually carried out by a computer mouse [16]. The proposed method checks which eye was closed and after what time it was re-opened. Based on this, the application ensures implementing the appropriate mouse event. Such a concept may prove useful for solutions with a relatively limited field of user activity, for example, mobile phones or other systems, the interface of which contains small components.

Interaction by closing both eyes involves the need to distinguish the involuntary blink from a consciously initiated action. In the literature, this topic is often referred to as *the Midas Touch Problem* [7, 13]. The authors of the blink-eye keyboard discussed earlier [15] solved this problem by assuming that the command directed to the system is expressed by closing the eyes for more than 200ms. According to the research, this is about twice as long as the average unconscious blink (100ms), which should allow for the correct interpretation of the gesture made by the eyes.

2.3. Eye Movement-based Interaction

Another kind of interaction can be realized by relatively simple eye movement sequences. Only a correct repeating the defined configuration leads to the execution of the specified operation. This solution has been used, among others, to enable switching between different modes used to control a computer game [11]. The research [13] has shown that performing this kind of – not too complicated – gestures may not only prevent unwanted selection of objects but may also be faster than some gaze-based methods. Additionally, there may be a limitation regarding the time in which the user can finish the started gesture [8]. The main disadvantage of this type of application control is the necessity of memorizing sequences initiating appropriate actions.

The research results and users' opinions on the tested eye-controlled systems show that the presented methods are promising [10, 13], but require further improvements, corrections, and tests [8, 11, 17]. It was the motivating factor to undertake current research and extend the set of possible solutions utilizing eyes as an input modality. All previously presented approaches for eye-based application management were considered and studied, which resulted in new solutions. Additionally, to the best of our knowledge, this paper is the first study comparing the above-described approaches.

3. Method

The first step of the presented studies was to elaborate solutions for the previously mentioned strategies. Their realisations are described in the following sections.

3.1. Gaze-based Control

The gaze-based control method assumes that controlling the application requires gathering not less than n samples. Until the value of n is reached, the algorithm's operation is limited only to data collection. Subsequently, the number of samples related to each object under consideration is calculated. The ratio of this value k to the n most recently recorded samples is compared with the earlier defined threshold (eq. 1).

$$\frac{k}{n} \geq threshold \tag{1}$$

An additional criterion that must be met to consider the component selected is the elapse of a specific time interval from the last element choice. This solution is designed to protect the element against its re-selection in a situation when the user did not manage to withdraw his gaze from the analysed area. When the procedure of selecting the item is completed, the recording process starts all over again. Information about the samples collected so far is deleted because they are related to the action already performed and should not affect the further operation. Then, at least n eye positions must be registered to resume the user's intentions detection.

The main problem of such a solution is the need to resolve how long the user's gaze must focus on one object to be successful, and the choice was deemed intentional. The more prolonged look at a given area of the screen allows selection with greater certainty that the action execution gave the desired effect. However, such control may prove tedious in the long run because it is relatively slow, with a lot of user work. Taking actions with a short gaze duration enables dynamic control of the application and quick initiation of further steps. On the other hand, this increases the risk that users, temporarily pausing their eyes or staring at items near the interactive object, will inadvertently make its selection. Of course, the best solution to this situation seems to compromise the need to keep the gaze on the selected element a little longer with a lower probability of performing unwanted operations. It may depend on the

user's preferences, which should be possible to configure in the application. In this research, the number of required samples n was set to 40 (at the sampling rate of 60Hz) and *threshold* to 95%. Additionally, it seems to be a good idea to provide an option that will allow withdrawing from the last action.

3.2. Blink-based Control

Due to the previously discussed difficulties related to distinguishing intentional blinking from the involuntary one, it was decided to focus only on gestures consisting in squinting only one eye. Thus, to perform an action, a user has to close the left or right eye, leaving the other eye open. Such a solution is possible when the eye tracker provides recordings for each eye separately. The realization of this eye gesture is visible in the eye-tracker signal as a series of samples (0, 0) obtained for a closed eye, whereas recordings for the other eye are different from zero. However, a user may unconsciously close the second eyelid, and the eye tracker recording will also be a point (0, 0). In order to improve the reading of the user's intention, a certain margin of tolerance has been introduced. The proposed solution discovers the occurrence of a gesture if k out of n analyzed samples contained a point (0, 0) for the same eye and a point other than (0, 0) for the second one. After the selection procedure is completed, the sampling process starts from scratch. However, if n samples were collected and no gestures are detected, the oldest one is no longer relevant and is deleted. In these studies, based on the preliminary processing, n was set to 30 and *threshold* to 80%.

3.3. Gesture-based Control

The considerations regarding methods based on executing fixed sequences allowed for the elaboration of two different concepts. The first one is based on points displayed on the screen, whereas the second approach relies on saccades and requires only the appropriate motion with the eyes.

3.3.1. Selecting the Point Sequence

This method utilizes a certain number of points by gazing at which, in short intervals of time and specific order, invokes an appropriate action. Connections between successive points are not visible on the screen. The person using the application focuses primarily on "drawing" a given shape with eyes rather than remembering the sequence. The proposed solution is similar to the directed graph. A single gesture can be defined by directed edges created between consecutive vertices.

When collecting data samples from the eye tracker, it is necessary to verify that a user has pointed out any of the displayed points and create a history of successively chosen elements. The method *Gaze-based Control* described in 3.1 may be used to select subsequent points. They are remembered together with their timestamps. The information recorded in this way is compared with the sequences that define the available gestures.

The method of connecting points requires adding just a few small buttons in the program window. It allows defining a vast set of actions that can be performed in the application. However, it may be challenging to memorize the order of the points in the sequence. Additionally, when the gesture fails, the user is forced to repeat it from scratch. Thus, applying the same parameters as for the *Gaze-based Control* method can be inconvenient and time-consuming; the user would have to spend about k times more time on performing a single gaze to obtain the same action. For this reason, it was decided to facilitate selecting the gesture elements by shortening a required gaze duration to 150ms. The element size was set to 90×90 pixels with a text size equal to 20 pixels. Additionally, 1000ms was applied as the acceptable interval between the gesture points selection.

3.3.2. Drawing the Gesture With the Gaze

In the following method, users perform a sequence of movements with their eyes to realize an action. However, this method does not require any additional graphical interface elements. The user 'draws' the defined sequence – similarly to the method discussed previously – but without looking at the specific points. There is more freedom and the possibility of providing a symbol in a convenient place. In such situations, the user's eyes pause for a short time, which leads to a fixation. Therefore, assessing whether the gesture has occurred relies on detecting fixations and determining their mutual location.

Due to the specificity of the eye's work and limited eye trackers accuracy, repeating precisely the same x or y fixation coordinates and obtaining a shape identical to the pattern is challenging: shapes will never be perfect and



Fig. 1. Displacement vector example.

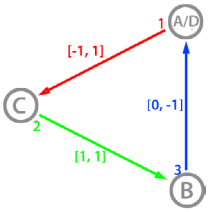


Fig. 2. Move to the left.

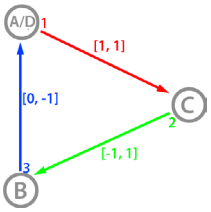


Fig. 3. Move to the right.

symmetrical. For this reason, it has to be assumed that the user's task is to make a shape close to the pattern. However, to assign fixations to a given gesture, the difference between their coordinates has to be less than the defined *deviation*. It is especially important for vertical or horizontal movements.

The decision if the user made the gesture is determined by examining the relationship between registered fixations. The shift vectors $[\Delta x, \Delta y]$ are defined for the consecutive pairs: $P_{f1} = (x_{f1}, y_{f1})$ and $P_{f2} = (x_{f2}, y_{f2})$, where P_{f1} is registered before P_{f2}. The displacement between them is calculated according to the equations 2:

$$\begin{aligned} dx &= x_{f2} - x_{f1} \\ dy &= y_{f2} - y_{f1} \end{aligned} \tag{2}$$

Then the values of the shift vector components are determined: $\Delta x = f(dx)$ and $\Delta y = -f(dy)$. The value of $f(dy)$, is preceded by a minus sign because the screen coordinates for the vertical axis increase from top to bottom (unlike the Cartesian coordinate system). The f function is defined as follows:

$$f(a) = \begin{cases} 1, & (a > 0) \wedge (|a| > deviation) \\ -1, & (a < 0) \wedge (|a| > deviation) \\ 0, & otherwise \end{cases} \tag{3}$$

The displacement vector obtained in this way allows determining if and in which direction the eye movement took place. An example situation is illustrated in the Fig.1. Two fixations were identified: A and B, which are at some distance from each other. Assuming that the point A was recorded first, the shift of gaze towards the area denoted as B is considered. Therefore, the vertical *(dy)* and horizontal *(dx)* differences between the fixations coordinates are calculated. If *dx* is less than or equal to the specified *deviation* value, the movement is considered as vertical. On the other hand, *dy* not greater than the *deviation* value allows assuming the existence of the same x coordinate.

Example sequences that may lead to two actions are presented in Fig. 2 and 3. The circles marked in the diagram with the symbols $A - D$ visualize the fixation points which are not displayed on the application screen. In the left case, the user starts the gesture by looking at any point located relatively far from the screen's bottom edge. A fixation (*A*) is detected. Subsequently, the user has to move vertically downwards and look for a short time (about 150 milliseconds). The subsequent fixation (*B*) will be identified. Then looking slightly upward, to the left, fixation (C) is obtained. The last stage is to look back to the place from which the execution of the gesture began. Similar steps regard the sequence presented in Fig. 3. As described earlier, the movement made with the eyes is interpreted by the algorithm based on the relative fixation positions. Using the formulas 2 and 3 the displacement vectors are determined for the successive pairs of registered fixations. For (A, B) the vector $[0, -1]$ is obtained, for (B, C): [-1, 1] and for (C, D): [1, 1]. The definition of such a gesture should therefore be given as a sequence of three vectors: $\{[0, -1]; [-1, 1]; [1, 1]\}$. In the proposed solution, the I-DT algorithm [18] for identifying fixations was used and *deviation* was set to 100 pixels.

4. The Experiment and Obtained Results

The above-described methods were verified with the usage of a test application implemented in the form of a simple photo viewer. The same activities were carried out in several ways, making the comparison of the methods possible. The application windows do not differ from the interfaces controlled by the mouse. Interactive elements can still be utilized by traditional methods. The application starts with the calibration utilizing jumping point as a stimulus. After that, a menu (Fig. 4) that allows the user to launch the photo viewer (Fig. 6, 7) or the settings panel (Fig. 5) is

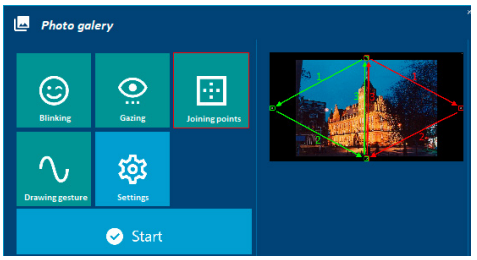


Fig. 4. The application main menu.

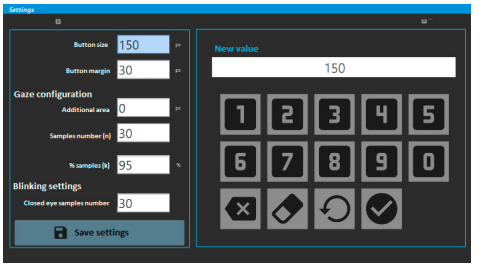


Fig. 5. Settings window.



Fig. 6. Blinking-based selection. The button visible under the icons were used in the gaze-based selection.



Fig. 7. Point-based gesture.

presented. There are five square buttons, selecting which is done by gaze. A red border appears around a pointed-out object. A description and gestures appropriate for the currently selected variant are displayed on the right.

The Eye Tribe eye tracker was used to carry out the tests working with a sampling rate equal to 60Hz. Several test scenarios were prepared, providing the possibilities of assessing methods taking various aspects into account. The application interface was displayed on the monitor screen, which dimensions were equal to 34cm and 19.5cm, and the resolution was set to 1920px×1080px. The user was sitting approximately 50cm from the eye tracker and about 70 from the screen. Eight people aged 20–30 participated in the research. All of them were no acquainted with eye-tracking technology. Three sessions were organized for each person within three consecutive days. Particular attention was paid to the calibration quality and a comfortable body position not to change it during the experiment.

The main experiment scenario was preceded by an initial one aimed at learning the environment and application. It lasted about 15 minutes. Subsequently, tasks testing the application's capabilities, including all four methods designed, were realized. At first, the navigation between photos was tested using the *Gaze-based Control* method and buttons with different dimensions. The settings screen was used to change their parameters, also with the eyes. Then, the navigation between the photos was done using the *Blinking-based Control* method. The user task was to find some details or poorly visible elements in the photo, which could lead to an unintentional gesture. After that, scrolling photos by connecting points and drawing gestures were tested similarly. The objects to be selected by the user were not chosen accidentally. Their layout might cause the necessity of a dynamic look at them, possibly to be interpreted as an unwanted gesture occurrence. All experiment steps took approximately 45 minutes. The order of the methods used was different for each session.

4.1. The Element Size Influence On the Gaze-based Selection Performance

When verifying the influence of the button size on the effectiveness of its selection, three different values were used: 150, 200, and 225 pixels. The participants had to focus on objects often placed near the left or right edge of the photo, close to buttons enabling navigation in the application. This situation could lead to an unintentional change of the photo. In such a case, the participant had to go back and complete the current task. When the situation repeated three times, the task was treated as uncompleted.

The obtained results are presented in Table 1. It can be noticed that a button of a size 200px, in the first session, was accidentally selected up to three times. The twice unwanted selections happened only once for 225px. A situation

in which a given user chose the button mistakenly once was observed ten times when three sessions are taken into consideration. All selections regarded two dimensions – 200 and 225px. The results also show that the number of incorrectly pointed-out buttons decreased with subsequent attempts. It may indicate that users have adjusted with time to the eye movement usage for controlling the application.

Table 1. The number of unintentionally selected buttons for each user and each attempt.

	The number of unintentionally selected buttons								
	Session I			**Session II**			**Session III**		
User	150px	200px	225px	150px	200px	225px	150px	200px	225px
#1	0	1	0	0	0	0	0	0	0
#2	0	0	0	0	0	1	0	0	0
#3	0	0	0	0	0	1	0	0	1
#4	0	0	0	0	0	1	0	0	1
#5	0	0	1	0	0	0	0	0	0
#6	0	1	2	0	0	1	0	0	0
#7	0	3	0	0	0	0	0	0	0
#8	0	0	1	0	0	0	0	0	0

During this step, the difficulty of selecting a button depending on its dimension was also explored. The participants were expected to initiate an action gazing at the appropriate element. If their intentions were not correctly recognized for approximately 1–1.5 seconds, they should remove their gaze from the button and look at the center of the screen. This action was a clear signal of a failed attempt. After a while, the user undertook the same action, making another attempt. For the analysis purpose, three selections of a specific button size were used in the consecutive attempts. It means that to switch between three photos, the participant had to press the same button at least three times. Based on the obtained results, the improvement in subsequent attempts was discovered, as in the previous task. It is represented by the increasing acceptance of the user's gesture during the first attempt. For example, in the most challenging size equal to 150px, there were 14 additional trials in the first attempt, whereas, in the second one, it was 9, and in the third, only 4.

The averaged outcomes for both test parts are shown in Table 2. The high values for standard deviation result from the fact that some subjects had many mistakes, and some of them performed the task without any failure.

Table 2. Averaged results for the unintended and unsuccessful selections of buttons of the given size.

	150px	200px	225px		150px	200px	225px
Avg number of unintended selections	0.00	0.63	1,25	Avg failure selection number	3.375	1	0.125
Standard deviation	0.00	1.06	1.04	Standard deviation	2.722	1.414	0.354

Additionally, to point out a component size, which could satisfy both analysed requirements – the low number of unwanted choices and the high performance in first selection – the results from both test parts were collated in the form of the chart presented in Fig. 8, on the left. It revealed that elements of size 200px provided results acceptable when unintended and effective selection numbers are taken into account. This finding was verified by asking users for their opinion. Half of them pointed one the same size. However, there were 37% users for which 150px was preferable and 13% voting for 225px.

4.2. The Other Methods Performance

A similar analysis to the above described was applied for the remaining set of gestures: blinking, connecting points, and drawing particular shapes. To provoke the possibility of unwanted gestures, the participant's gaze was directed to perform a sequence as resembling as possible to the gestures defined in the application. For each of the methods, three such combinations were developed, adjusted to the photos presented. In some cases, this type of action led to an

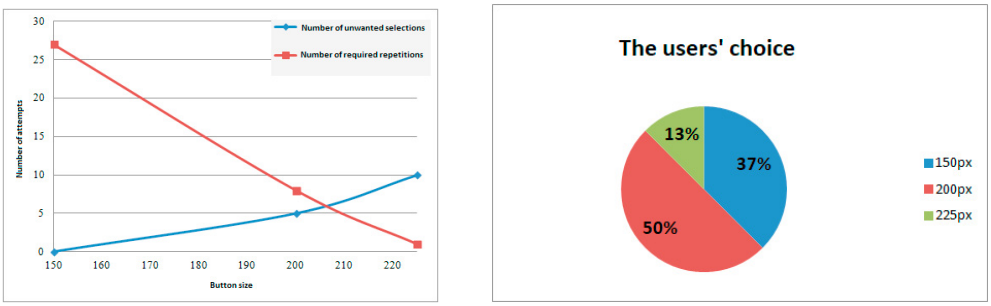


Fig. 8. Efficient object selection and the number of unwanted actions depending on the component size (on the left). The best size of the switching button according to participants' opinions (on the right).

inadvertent image switching. This situation did not require users to go back to the recently viewed photo. They could move on to the next point of the test scenario. It was observed that each participant performed an unintended action at least once. However, avoiding it was the easiest with the usage of connecting points approach. It also turned out that one of the participants was unable to use the eye blink method to invoke photo change.

The application also verified a much simpler sequence, consisting of only one connection of two buttons (in any order). These points are placed in the photo gallery window, enabling closing it with the connecting points method, regardless of the currently utilized approach. The buttons were selectable for the entire time the application was launched, thus creating a risk that the participant – during the test scenario – would inadvertently make the gesture of closing the application. It turned out that despite the over 45 minutes experiment duration, an accidental triggering of this gesture happened only 7 times, resulting in the closing of the gallery window.

The proposed methods were not always able to determine the user's intentions after the first gesture trial. In order to investigate this problem, the participants of the experiment were asked to dynamically switch photos – three forward and three back – for each of the four methods, making 24 gestures (6 for each approach).

The results achieved in this task varied for participants depending on which attempt to the experiment it was. Additionally, some users found some methods more difficult than others. Once again, one participant was unable to utilize the blink-based method. The percentage of the gestures made correctly with the first attempt (FA) together with the average number of attempts (ANA) to perform a single operation, for all defined methods is presented in Table 3.

Table 3. Percentage of the gestures correctly identified during first attempt (FA) and average number of attempts (ANA)

	Gaze-based control		**Blinking-based control**		**Gesture-based control**		**Connecting points**	
	FA	ANA	FA	ANA	FA	ANA	FA	ANA
Attempt I	87.50%	1.15	69.05%	1.31	33.33%	1.75	89.58%	1.10
Attempt II	97.92%	1.02	59.52%	1.40	58.33%	1.63	95.83%	1.06
Attempt III	81.25%	1.25	73.81%	1.15	66.67%	1.33	85.42%	1.17
Average:	88.89%	1.14	67.46%	1.28	52.78%	1.57	90.28%	1.11

4.3. The Gesture Speed Dependency on the Method Used

Another analyzed aspect was the time required to perform a single operation using elaborated methods. In this part of the experiment, the user's attention was focused only on switching the photos visible in the application window: forward three and then back three.

When comparing the outcomes presented in Table 4 with these shown in Table 3, their dependence on the necessity of repeating the gesture can be observed. In most cases, it had a negative impact on the task duration. During the experiment, it was noticed that the participants made some gestures at different speeds, and some of them were unable to perform the dynamic eye movements properly. Based on the obtained results, the average time needed to take a

Table 4. Time, in seconds, required for switching six photos obtained for different methods: GZ - gaze-based, BL- blinking-based, CP - connecting points, GS - drawing gesture

	Session I				Session II				Session III			
	GZ	BL	CP	GS	GZ	BL	CP	GS	GZ	BL	CP	GS
Avg.	15.08	26.28	21.54	42.97	11.54	25.86	20.46	32.38	16.22	15.62	21.40	25.11
StDev.	4.12	13.18	4.70	14.46	4.85	9.79	3.37	11.49	9.93	3.51	4.75	4.45

Table 5. Assessment of the selection methods performed by the participants.

Participant Id	GZ	BL	GS	CP
#1	3	4	2	5
#2	3	4	3	5
#3	5	4	1	3
#4	4	2	5	4
#5	3	2	3	4
#6	5	—	3	4
#7	4	4	2	5
#8	4	1	3	5
Avg.:	3.88	3.00	2.75	4.38
StDev.	0.83	1.29	1.16	0.74

single action was determined. It was 2.38s for gaze-based method, 3.52s for connecting points, 3.74s for blinking and 5.58s for drawing gestures.

4.4. The Participants' Assessment of the Methods

After completing the third sessions, participants were asked to evaluate the methods they tested. They were expected to assign a mark from 1 (the lowest) to 5 (the highest) for each of them. The assessment was based not only on the results achieved by a given person but also on one's general impression of a given solution – the comfort of work, difficulty, and the effort put into learning the approach. The assigned grades are collected in Table 5.

They reveal that the method based on connecting points was the most appreciated by the research group. Users pointed out the intuitiveness, speed, and low unreliability as this approach advantages. The average rating indicated that the participants were the least satisfied utilizing gesture drawing by eye movements. According to the participants' comments, it was a significant problem to reproduce the correct shape, the contours of which were not visible on the screen. The blinking gestures received different opinions. According to some participants, they required remarkable concentration, and at the beginning, it was related to some discomfort. However, these opinions should be confirmed with a more extensive group of users.

5. Summary

In the paper, the new solutions for controlling computer programs with eyes were discussed. Among them, gaze-, blinking- and gestures-based approaches were considered. Their performance was compared with the usage of a simple photo viewer. Although the designed gestures were utilized for switching photos, they can be implemented in other applications. Changing e-book pages or scrolling some types of on-screen keyboards may be mentioned here as examples. Moreover, by a simple program extension by adding two buttons at the top and bottom of the screen as well as two gesture types, new directions of the movement can be obtained – to the top and the bottom.

The achieved results allowed pointing out two most effective methods – gaze-based and connecting points approaches. The participants' assessment verified this finding. They filled a questionnaire in which they referred to their general feelings regarding each method. It should be emphasized that in this research, the cheap eye tracker was used – affordable for many users. Therefore, it can be expected that by utilizing more expensive devices, more precise

gaze point estimation could be achieved, entailing a lower error rate in the performed actions. I will also influence the number of samples n used for determining gestures.The application was equipped with a module for customizing the methods' settings, making them more adjusted and convenient for a particular person. Additionally, when utilizing the application, a user can choose a preferable approach for the application control, which currently is able to recognize several gestures; however, new ones are planned as future work.

Acknowledgements

This research was funded by the Statutory Research funds of Faculty of Automatic Control, Electronics and Computer Science, Grant Number 02/100/BK_21/0008.

References

[1] Y.-C. Jian, H.-W. Ko, Influences of text difficulty and reading ability on learning illustrated science texts for children: An eye movement study, Computers & Education 113 (2017) 263–279.
[2] A. Gegenfurtner, E. Lehtinen, H. Jarodzka, R. Säljö, Effects of eye movement modeling examples on adaptive expertise in medical image diagnosis, Computers & Education 113 (2017) 212–225.
[3] S. L. Rogers, C. P. Speelman, O. Guidetti, M. Longmuir, Using dual eye tracking to uncover personal gaze patterns during social interaction, Scientific reports 8 (1) (2018) 1–9.
[4] K. Harezlak, J. Rzeszutek, P. Kasprowski, The eye tracking methods in user interfaces assessment, in: R. Neves-Silva, L. C. Jain, R. J. Howlett (Eds.), Intelligent Decision Technologies, Springer International Publishing, Cham, 2015, pp. 325–335.
[5] Z. Xuebai, L. Xiaolong, Y. Shyan-Ming, L. Shu-Fan, Eye tracking based control system for natural human-computer interaction, Computational Intelligence and Neuroscience 2017 (2017) 1–9.
[6] F. Onur, V. Fernando, Low cost eye tracking: The current panorama, Computational Intelligence and Neuroscience 2016 (2016) 1–14.
[7] P. Kasprowski, K. Harezlak, M. Niezabitowski, Eye movement tracking as a new promising modality for human computer interaction, in: 2016 17th International Carpathian Control Conference (ICCC), 2016, pp. 314–318.
[8] V. Vaitukaitis, A. Bulling, Eye gesture recognition on portable devices, UbiComp'12 - Proceedings of the 2012 ACM Conference on Ubiquitous Computing (2012) 711–714.
[9] M. Casarini, M. Porta, P. Dondi, A gaze-based web browser with multiple methods for link selection, in: ACM Symposium on Eye Tracking Research and Applications, ETRA '20 Adjunct, Association for Computing Machinery, New York, NY, USA, 2020. doi:10.1145/3379157.3388929.
[10] A. M. Penkar, C. Lutteroth, G. Weber, Designing for the eye: Design parameters for dwell in gaze interaction, in: Proceedings of the 24th Australian Computer-Human Interaction Conference, 2012, pp. 479–488.
[11] H. Istance, R. Bates, A. Hyrskykari, S. Vickers, Snap clutch, a moded approach to solving the midas touch problem, Eye Tracking Research and Applications Symposium (ETRA) (2008) 221–228.
[12] S. Linda, J. Robert, Evaluation of eye gaze interaction, in: CHI '00. The Hague, The Netherlands: ACM, 2000, pp. 281–288.
[13] H. Aulikki, I. Howell, V. Stephen, Gaze gestures or dwell-based interaction?, in: Proceedings of the Symposium on Eye Tracking Research and Applications, ETRA '12, 2012, pp. 229–232.
[14] M. Emiliano, W. Tianyu, A. T. Campbell, Eyephone: Activating mobile phones with your eyes, in: ACM Press, 2010, pp. 15–20.
[15] A. Krolak, P. Strumillo, Eye-blink detection system for human–computer interaction, Universal Access in the Information Society 11 (2011) 1–11.
[16] P. Kowalczyk, D. Sawicki, Blink and wink detection as a control tool in multimodal interaction, Multimedia Tools and Applications (2018).
[17] K. Raju, P. Haripriya, Eye gaze tracking with a web camera in a desktop environment, International Journal of Research 3 (13) (2016) 1230–1235.
[18] D. D. Salvucci, J. H. Goldberg, Identifying fixations and saccades in eye-tracking protocols, in: Proceedings of the 2000 Symposium on Eye Tracking Research & Applications, ETRA '00, Association for Computing Machinery, New York, NY, USA, 2000, p. 71–78. doi:10.1145/355017.355028.